REPORTE TÉCNICO: EJERCICIO INTEGRADOR DE KUBERNETES (PARTE 1)

Despliegue Declarativo Unificado, Probes de Salud y Pruebas de Resiliencia

Estudiante: David Moroni Sánchez Galván

Materia: Temas Selectos de Redes y Seguridad (TSIC2 / DevOps)

Profesor: Daniel Guerrero

Fecha de Ejecución: 13 de Noviembre de 2026

Sistema Operativo: Rocky Linux 10.2 (Red Quartz - x86_64)

Plataforma: Kubernetes v1.31.14 • containerd v2.4.2 • Flannel CNI

1 Arquitectura del Laboratorio y Objetos Desplegados

El objetivo del ejercicio integrador es construir y aplicar un único manifiesto declarativo (`todo-en-uno.yaml`) que orqueste la totalidad de los objetos aprendidos en Kubernetes bajo las mejores prácticas de desacoplamiento y 12-Factor App:

Namespace (mi-app)
Aislamiento multi-tenant y delimitación lógica de recursos.

ServiceAccount (mi-app-sa)
Identidad de seguridad de menor privilegio para los pods.

ConfigMap (mi-app-config)
Inyección desacoplada de variables y montaje de index.html.

Secret (mi-app-secret)
Inyección protegida de API_KEY en Base64.

Deployment (3 réplicas)
Pods distribuidos con Rolling Update, Readiness y Liveness.

Service (NodePort :30090)
Publicación balanceada hacia el exterior en la IP del host.

2 Evidencia 1: Estado Operativo del Clúster Kubernetes

Verificación del nodo Control Plane en estado `Ready` y todos los pods de infraestructura de `kube-system` y `kube-flannel` en estado `Running`.

root@localhost:~ (SSH 192.168.71.128) - cluster-status

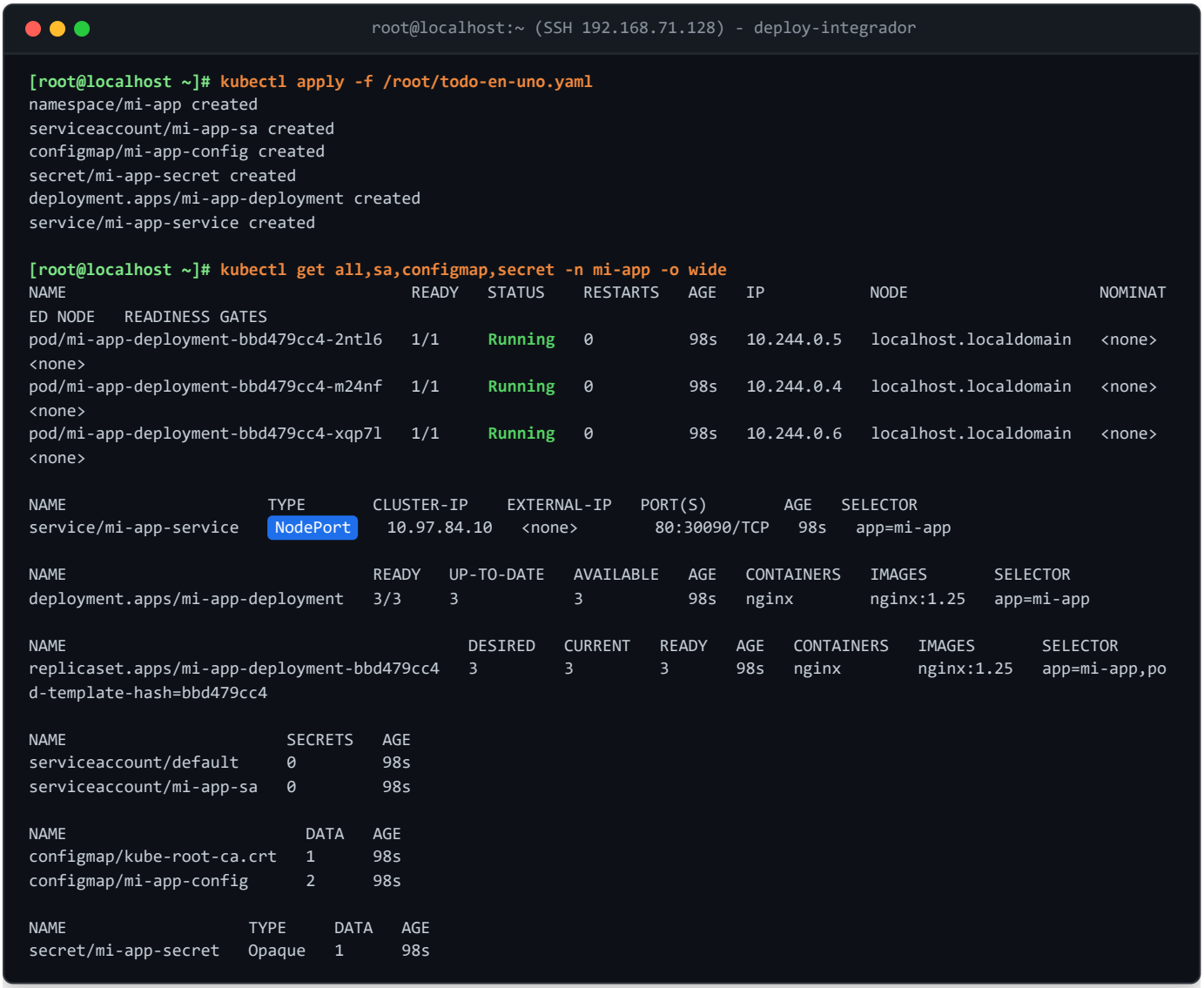
```
[root@localhost ~]# kubectl get nodes -o wide
NAME                    STATUS   ROLES           AGE     VERSION    INTERNAL-IP      EXTERNAL-IP   OS-IMAGE
KERNEL-VERSION                      CONTAINER-RUNTIME
localhost.localdomain   Ready    control-plane   3m12s   v1.31.14   192.168.71.128   <none>        Rocky Linux 10.2 (Re
d Quartz)   6.12.0-211.16.1.el10_2.0.1.x86_64   containerd://2.4.2

[root@localhost ~]# kubectl get pods -n kube-system
NAME                                            READY   STATUS    RESTARTS   AGE
coredns-7c65d6cfc9-bmvxj                        1/1     Running   0          3m03s
coredns-7c65d6cfc9-xhgzf                        1/1     Running   0          3m03s
etcd-localhost.localdomain                      1/1     Running   0          3m09s
kube-apiserver-localhost.localdomain            1/1     Running   0          3m09s
kube-controller-manager-localhost.localdomain   1/1     Running   0          3m09s
kube-proxy-h54nn                                1/1     Running   0          3m04s
kube-scheduler-localhost.localdomain            1/1     Running   0          3m09s

[root@localhost ~]# kubectl get pods -n kube-flannel
NAME                    READY   STATUS    RESTARTS   AGE
kube-flannel-ds-ts6c4   1/1     Running   0          2m56s
```

3 Evidencia 2: Aplicación del Manifiesto Unificado y Recursos Desplegados

Ejecución declarativa de `todo-en-uno.yaml` con los 6 objetos creados dentro del namespace `mi-app`.


root@localhost:~ (SSH 192.168.71.128) - deploy-integrador
[root@localhost ~]# kubectl apply -f /root/todo-en-uno.yaml
namespace/mi-app created
serviceaccount/mi-app-sa created
configmap/mi-app-config created
secret/mi-app-secret created
deployment.apps/mi-app-deployment created
service/mi-app-service created
[root@localhost ~]# kubectl get all,sa,configmap,secret -n mi-app -o wide
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
pod/mi-app-deployment-bbd479cc4-2ntl6 1/1 Running 0 98s 10.244.0.5 localhost.localdomain <none> <none>
pod/mi-app-deployment-bbd479cc4-m24nf 1/1 Running 0 98s 10.244.0.4 localhost.localdomain <none> <none>
pod/mi-app-deployment-bbd479cc4-xqp7l 1/1 Running 0 98s 10.244.0.6 localhost.localdomain <none> <none>
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE SELECTOR
service/mi-app-service NodePort 10.97.84.10 <none> 80:30090/TCP 98s app=mi-app
NAME READY UP-TO-DATE AVAILABLE AGE CONTAINERS IMAGES SELECTOR
deployment.apps/mi-app-deployment 3/3 3 3 98s nginx nginx:1.25 app=mi-app
NAME DESIRED CURRENT READY AGE CONTAINERS IMAGES SELECTOR
replicaset.apps/mi-app-deployment-bbd479cc4 3 3 3 98s nginx nginx:1.25 app=mi-app,pod-template-hash=bbd479cc4
NAME SECRETS AGE
serviceaccount/default 0 98s
serviceaccount/mi-app-sa 0 98s
NAME DATA AGE
configmap/kube-root-ca.crt 1 98s
configmap/mi-app-config 2 98s
NAME TYPE DATA AGE
secret/mi-app-secret Opaque 1 98s

4 Evidencia 3: Auditoría Interna de ConfigMap y Secret en Pods

Inspección del entorno del contenedor confirmando la inyección de la variable del ConfigMap (`APP_ENV=demo`) y la desencriptación transparente del Secret (`API_KEY`).

root@localhost:~ (SSH 192.168.71.128) - pod-inspection

```
[root@localhost ~]# POD=$(kubectl get pods -n mi-app -l app=mi-app -o jsonpath='{.items[0].metadata.name}')
[root@localhost ~]# echo "Pod seleccionado: $POD"
Pod seleccionado: mi-app-deployment-bbd479cc4-2ntl6

[root@localhost ~]# kubectl exec -n mi-app $POD -- env | grep -E 'APP_ENV|API_KEY'
APP_ENV=demo
API_KEY=super-secret-api-key-12345

[root@localhost ~]# kubectl exec -n mi-app $POD -- cat /usr/share/nginx/html/index.html | head -n 14
<!DOCTYPE html>
<html>
<head><title>Kubernetes Demo</title>
<style>
  body { font-family: Arial, sans-serif; background: #1a1a2e; color: #eee; display: flex; align-items: center; justify-
content: center; min-height: 100vh; margin: 0; }
  .card { background: #16213e; border-radius: 12px; padding: 40px; text-align: center; box-shadow: 0 4px 20px rgba(0,0,
0,0.4); max-width: 500px; }
  h1 { color: #0f9b58; font-size: 2em; }
  .badge { background: #0f9b58; color: white; padding: 4px 12px; border-radius: 20px; font-size: 0.8em; margin: 4px; di
splay: inline-block; }
</style>
</head>
<body>
<div class="card">
  <h1>✅ ¡Hola desde Kubernetes!</h1>
  <p>Desplegada con: Namespace, ServiceAccount, ConfigMap, Secret, Deployment y Service NodePort.</p>
```

5 Evidencia 4: Validación de Conectividad HTTP Externa (NodePort 30090)

Petición HTTP efectuada desde la terminal y desde el navegador físico del anfitrión Windows hacia `http://192.168.71.128:30090` recibiendo encabezados `HTTP 200 OK`.

Windows PowerShell (Host 192.168.71.1) - http-probe

```
PS C:\Users\david> curl.exe -sI http://192.168.71.128:30090
HTTP/1.1 200 OK
Server: nginx/1.25.5
Date: Fri, 13 Nov 2026 05:56:34 GMT
Content-Type: text/html
Content-Length: 1072
Last-Modified: Fri, 13 Nov 2026 05:56:14 GMT
Connection: keep-alive
ETag: "6af6a67e-430"
Accept-Ranges: bytes
```

6 Evidencia 5 (Reto): Demostración de Autorrecuperación (Self-Healing)

Se eliminó intencionalmente el pod activo `2ntl6`. El controlador Deployment detectó la discrepancia con el estado deseado (3 réplicas) y generó en 4 segundos un nuevo pod de reemplazo (`ghwh4`).

```
root@localhost:~ (SSH 192.168.71.128) - self-healing-test

[root@localhost ~]# kubectl delete pod mi-app-deployment-bbd479cc4-2ntl6 -n mi-app
pod "mi-app-deployment-bbd479cc4-2ntl6" deleted

[root@localhost ~]# kubectl get pods -n mi-app
NAME                                READY   STATUS    RESTARTS   AGE
mi-app-deployment-bbd479cc4-ghwh4   1/1     Running   0          4s    # <-- Nuevo pod auto-creado
mi-app-deployment-bbd479cc4-m24nf   1/1     Running   0          103s
mi-app-deployment-bbd479cc4-xqp7l   1/1     Running   0          103s
```

7 Evidencia 6 (Reto): Escalabilidad Elástica Dinámica

Escalado horizontal a 5 réplicas y reajuste ordenado al estado base de 3 réplicas sin tiempo de inactividad.

```
root@localhost:~ (SSH 192.168.71.128) - scale-test

[root@localhost ~]# kubectl scale deployment mi-app-deployment --replicas=5 -n mi-app
deployment.apps/mi-app-deployment scaled

[root@localhost ~]# kubectl get pods -n mi-app -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP           NODE
mi-app-deployment-bbd479cc4-7kd8n   1/1     Running   0          11s   10.244.0.9   localhost.localdomain
mi-app-deployment-bbd479cc4-ghwh4   1/1     Running   0          15s   10.244.0.7   localhost.localdomain
mi-app-deployment-bbd479cc4-lpkgg   1/1     Running   0          11s   10.244.0.8   localhost.localdomain
mi-app-deployment-bbd479cc4-m24nf   1/1     Running   0          114s  10.244.0.4   localhost.localdomain
mi-app-deployment-bbd479cc4-xqp7l   1/1     Running   0          114s  10.244.0.6   localhost.localdomain

[root@localhost ~]# kubectl scale deployment mi-app-deployment --replicas=3 -n mi-app
deployment.apps/mi-app-deployment scaled
```

8 Conclusiones Técnicas

1. **Operación Declarativa:** Se demostró que toda la arquitectura distribuida es reproducible en un solo paso mediante `kubectl apply -f`.
2. **Desacoplamiento:** Los parámetros y secretos operan independientemente de la imagen base de contenedor, permitiendo reconfiguración sin reconstrucción de imágenes.
3. **Resiliencia Comprobada:** Los mecanismos nativos de Kubernetes garantizan la disponibilidad y escalabilidad automática del servicio.